OOP FINAL

BSFT

Question 1

You are tasked with designing a program to simulate a stock exchange system. In this system, the following requirements must be fulfilled.

1)Create a class StockExchange with the following attributes:

- listed_companies (a list of companies listed on the stock exchange).
- share_value (a dictionary storing each company's current share value).
- share_price (a dictionary storing each company's current share price).

2) Create a class Investor with the following attributes:

- name (name of the investor).
- portfolio (a dictionary storing the shares of companies the investor owns, with company names as keys and the number of shares as values).
- balance (the investor's available balance for trading).

3) Implement the following methods:

- In the StockExchange class:
- buy_shares(investor, company, quantity): Allows an investor to buy a specific number of shares of a company. Deduct the total cost from the investor's balance and update their portfolio.
- sell_shares(investor, company, quantity): Allows an investor to sell shares of a company. Add the proceeds to the investor's balance and update their portfolio.
- transfer_shares(investor_from, investor_to, company, quantity): Transfers a specific number of shares of a company from one investor to another.
- In the Investor class:
- display_portfolio(): Displays the investor's portfolio and balance.

4) Demonstrate the following scenario:

- Two investors, Alice and Bob, participate in the stock exchange.
- Alice buys 50 shares of "TechCorp" and 30 shares of "BioHealth."

- Bob buys 20 shares of “TechCorp.”
- Alice transfers 10 shares of “TechCorp” to Bob.
- Bob sells 5 shares of “TechCorp.”
- Display the portfolios and balances of Alice and Bob at the end of the simulation.

Requirements:

- Demonstrate the concepts of classes, attributes, methods, encapsulation, and object interactions.
- Ensure that all operations handle edge cases (e.g., insufficient balance or shares).
- Write the full implementation in Python.

CODE:

```
class StockExchange:
    def __init__(self):
        self.listed_companies = ["TechCorp", "BioHealth"]
        self.share_value = {"TechCorp": 100, "BioHealth": 150}
        self.share_price = {"TechCorp": 10, "BioHealth": 15}

    def buy_shares(self, investor, company, quantity):
        if company not in self.listed_companies:
            print(f"{company} is not listed on the stock exchange.")
            return
        total_cost = self.share_price[company] * quantity
        if investor.balance >= total_cost:
            investor.balance -= total_cost
            investor.portfolio[company] = investor.portfolio.get(company, 0) + quantity
            print(f"{investor.name} bought {quantity} shares of {company}.")
```

```
        else:
            print(f"{investor.name} does not have enough balance to buy {quantity} shares of {company}.")

    def sell_shares(self, investor, company, quantity):
        if company in investor.portfolio and investor.portfolio[company] >= quantity:
            total_earnings = self.share_price[company] * quantity
            investor.balance += total_earnings
            investor.portfolio[company] -= quantity
            if investor.portfolio[company] == 0:
                del investor.portfolio[company]
            print(f"{investor.name} sold {quantity} shares of {company}.")
        else:
            print(f"{investor.name} does not have enough shares of {company} to sell.")

    def transfer_shares(self, investor_from, investor_to, company, quantity):
        if company in investor_from.portfolio and investor_from.portfolio[company] >= quantity:
            investor_from.portfolio[company] -= quantity
            if investor_from.portfolio[company] == 0:
                del investor_from.portfolio[company]
            investor_to.portfolio[company] = investor_to.portfolio.get(company, 0) + quantity
            print(f"{quantity} shares of {company} transferred from {investor_from.name} to {investor_to.name}.")
        else:
            print(f"{investor_from.name} does not have enough shares of {company} to transfer.")
```

```
class Investor:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance
        self.portfolio = {}

    def display_portfolio(self):
        print(f"\n{self.name}'s Portfolio:")
        for company, shares in self.portfolio.items():
            print(f"{company}: {shares} shares")
        print(f"Balance: ${self.balance}")



exchange = StockExchange()
alice = Investor("Alice", 2000)
bob = Investor("Bob", 1000)

exchange.buy_shares(alice, "TechCorp", 50)
exchange.buy_shares(alice, "BioHealth", 30)
exchange.buy_shares(bob, "TechCorp", 20)
exchange.transfer_shares(alice, bob, "TechCorp", 10)
exchange.sell_shares(bob, "TechCorp", 5)
```

alice.display_portfolio()

bob.display_portfolio()

Question 2:

Design a Snake and Ladder game using Object-Oriented Programming principles. The game should be implemented in Python and follow the requirements below:

1)Create a Board class that:

- Initializes a 10x10 board (100 squares).
- Contains dictionaries ladders and snakes where keys represent starting positions of ladders/snakes, and values represent their end positions.

2) Create a Player class that:

- Stores the player's name and current position (initially 1).
- Has a method move(dice_value) to update the player's position based on dice rolls.

3) Create a Game class that:

- Initializes the board and allows up to two players.
- Has a method roll_dice() that returns a random value between 1 and 6.
- Has a method play_turn(player) to:
- Roll the dice and move the player.
- Check if the player's new position lands on a ladder or snake and update the position accordingly.
- If the player lands on a ladder, they get another turn to roll the dice.
- Declare the winner when a player reaches or exceeds position 100.

4) Demonstrate the game by:

- Adding two players (e.g., "Alice" and "Bob").
- Simulating the game turn-by-turn until a winner is declared.

Requirements:

- Demonstrate object creation, method calls, conditional statements, and random module usage.
- Ensure that all game rules are implemented.
- Provide the full code implementation and output for a sample game simulation.
- Write the implementation in Python.

CODE:

```
import random


class Board:
    def __init__(self):
        self.ladders = {3: 22, 5: 8, 11: 26, 20: 29, 17: 4}
        self.snakes = {27: 1, 21: 9, 19: 7, 16: 3, 25: 10}


class Player:
    def __init__(self, name):
        self.name = name
        self.position = 1

    def move(self, dice_value):
        self.position += dice_value
        if self.position > 100:
            self.position = 100

    def update_position(self, new_position):
        self.position = new_position


class Game:
    def __init__(self):
```

```
        self.board = Board()
        self.players = []
        self.winner = None

    def add_player(self, player):
        self.players.append(player)

    def roll_dice(self):
        return random.randint(1, 6)

    def play_turn(self, player):
        print(f"\n{player.name}'s turn:")
        dice_value = self.roll_dice()
        print(f"Rolled a {dice_value}!")
        player.move(dice_value)

        if player.position in self.board.ladders:
            print(f"Hit a ladder at {player.position}! Climbing to {self.board.ladders[player.position]}.")
            player.update_position(self.board.ladders[player.position])
            print(f"Rolling again due to the ladder!")
            self.play_turn(player)  # Recursive call for another turn

        elif player.position in self.board.snakes:
            print(f"Bitten by a snake at {player.position}! Sliding to
{self.board.snakes[player.position]}.")
```

```
            player.update_position(self.board.snakes[player.position])

        print(f"{player.name} is now at position {player.position}.")

        if player.position == 100:
            self.winner = player
            print(f"\n🎉 {player.name} wins the game! 🎉")

    def start_game(self):
        print("Starting the Snake and Ladder game!")
        while not self.winner:
            for player in self.players:
                self.play_turn(player)
                if self.winner:
                    break

game = Game()
player1 = Player("Alice")
player2 = Player("Bob")

game.add_player(player1)
game.add_player(player2)

game.start_game()
```

Question 3

You are tasked with designing a program to simulate a transportation booking system. In this system, the following requirements must be fulfilled:

1)Create a base class Transportation with the following attributes:

- vehicle_type (the type of vehicle used, e.g., “Car”, “Bike”, etc.).
- capacity (the maximum number of passengers the vehicle can hold).
- base_fare (the starting fare for a trip).

2) Create three child classes Yango, Careem, and Uber that inherit from the Transportation class:

- Each child class must have its own unique fare_method to calculate the total fare for a trip based on the distance and time.

3) Implement the following methods:

- In the Transportation class:
- book_ride(distance, passengers): Books a ride for the given number of passengers and distance. It should check if the number of passengers exceeds the vehicle’s capacity and display an appropriate message.
- cancel_ride(): Cancels a booked ride and displays a confirmation message.
- display_details(): Displays the details of the transportation, including vehicle type, capacity, and base fare.
- In each child class (Yango, Careem, and Uber):
- Implement a unique fare_method:
- Yango: Calculate the fare as base_fare + 5 * distance.
- Careem: Calculate the fare as base_fare + 3 * distance + 2 * time.
- Uber: Calculate the fare as base_fare + 10 * distance - discount (where discount is 10% of the total distance fare for trips over 20 km).

4) Demonstrate the following scenario:

- A ride is booked with each service (Yango, Careem, and Uber) for a distance of 15 km and 2 passengers.
- A second ride is booked with Uber for a distance of 25 km and 4 passengers.
- Cancel the first ride with Careem.
- Display the details and fare calculations for all rides booked.
- Ensure that all operations handle edge cases (e.g., exceeding vehicle capacity or booking with invalid inputs).

Requirements:

- Demonstrate the concepts of inheritance, polymorphism, encapsulation, and method overriding.
- Ensure all methods handle edge cases (e.g., invalid passenger count, negative distance, etc.).
- Write the full implementation in Python.

CODE:

```
class Transportation:

    def __init__(self, vehicle_type, capacity, base_fare):

        self.vehicle_type = vehicle_type

        self.capacity = capacity

        self.base_fare = base_fare

        self.is_booked = False


    def book_ride(self, distance, passengers):

        if passengers > self.capacity:

            print(f"Booking failed: {self.vehicle_type} can only hold up to {self.capacity} passengers.")

            return False

        elif distance <= 0 or passengers <= 0:

            print("Booking failed: Invalid distance or passenger count.")

            return False

        else:

            self.is_booked = True

            print(f"Ride booked with {self.vehicle_type} for {passengers} passengers and {distance} km.")
```

```
            return True

    def cancel_ride(self):
        if self.is_booked:
            self.is_booked = False
            print(f"Ride with {self.vehicle_type} has been canceled.")
        else:
            print(f"No ride to cancel for {self.vehicle_type}.")

    def display_details(self):
        print(f"Vehicle Type: {self.vehicle_type}")
        print(f"Capacity: {self.capacity}")
        print(f"Base Fare: ${self.base_fare}")

class Yango(Transportation):
    def __init__(self, capacity, base_fare):
        super().__init__("Yango", capacity, base_fare)

    def fare_method(self, distance):
        return self.base_fare + 5 * distance

class Careem(Transportation):
    def __init__(self, capacity, base_fare):
        super().__init__("Careem", capacity, base_fare)
```

```
    def fare_method(self, distance, time):
        return self.base_fare + 3 * distance + 2 * time

class Uber(Transportation):
    def __init__(self, capacity, base_fare):
        super().__init__("Uber", capacity, base_fare)

    def fare_method(self, distance):
        distance_fare = 10 * distance
        discount = 0.1 * distance_fare if distance > 20 else 0
        return self.base_fare + distance_fare - discount


yango = Yango(capacity=4, base_fare=10)
careem = Careem(capacity=4, base_fare=12)
uber = Uber(capacity=6, base_fare=8)

print("\n--- Booking Rides ---")
if yango.book_ride(distance=15, passengers=2):
    print(f"Yango Fare: ${yango.fare_method(15)}")

if careem.book_ride(distance=15, passengers=2):
    print(f"Careem Fare: ${careem.fare_method(15, time=10)}")
```

```
if uber.book_ride(distance=15, passengers=2):
    print(f"Uber Fare: ${uber.fare_method(15)}")

if uber.book_ride(distance=25, passengers=4):
    print(f"Uber Fare for 25 km: ${uber.fare_method(25)}")

print("\n--- Cancel Careem Ride ---")
careem.cancel_ride()

print("\n--- Display Details ---")
yango.display_details()
careem.display_details()
uber.display_details()
```

Question 4

Question: Clock System Simulation

You are tasked with designing a program to simulate a clock system with altered time settings. The clock system works as follows:

1)Create a base class Clock with the following attributes:

- hours_per_day (total number of hours in a day for the clock).
- minutes_per_hour (number of minutes in each hour for the clock).
- current_time (stores the current time in hours and minutes).

2) Create two child classes ConventionalClock and AlteredClock that inherit from the Clock class:

- ConventionalClock: Represents the standard 24-hour clock system (12 hours AM/PM format and 60 minutes per hour).

- AlteredClock: Represents the alternate clock system with 12 hours per day and 30 minutes per hour.

3) Implement the following methods:

- In the Clock class:
- set_time(hours, minutes): Sets the current time to the given hours and minutes, ensuring it fits within the clock's configuration.
- add_minutes(minutes): Adds a specific number of minutes to the current time, updating the hours and minutes accordingly.
- display_time(): Displays the current time in the format HH:MM.
- In each child class:
- describe_clock(): Displays details about the clock's configuration (e.g., hours per day and minutes per hour).

4) Demonstrate the following scenario:

- Create a ConventionalClock instance and set the time to 10:30.
- Add 95 minutes to the ConventionalClock and display the updated time.
- Create an AlteredClock instance and set the time to 5:15.
- Add 50 minutes to the AlteredClock and display the updated time.
- Display the configurations of both clocks.

Requirements:

- Demonstrate the concepts of inheritance, method overriding, encapsulation, and object interactions.
- Ensure all methods handle edge cases (e.g., invalid time inputs or exceeding the clock's limits).
- Write the full implementation in Python.

CODE:

```
class Clock:

    def __init__(self, hours_per_day, minutes_per_hour):

        self.hours_per_day = hours_per_day

        self.minutes_per_hour = minutes_per_hour

        self.current_time = {'hours': 0, 'minutes': 0}
```

```
    def set_time(self, hours, minutes):
        if 0 <= hours < self.hours_per_day and 0 <= minutes < self.minutes_per_hour:
            self.current_time['hours'] = hours
            self.current_time['minutes'] = minutes
            print(f"Time set to {hours:02}:{minutes:02}")
        else:
            print(f"Invalid time! Hours must be between 0 and {self.hours_per_day - 1}, "
                  f"and minutes must be between 0 and {self.minutes_per_hour - 1}.")

    def add_minutes(self, minutes):
        total_minutes = self.current_time['hours'] * self.minutes_per_hour +
self.current_time['minutes'] + minutes
        new_hours = (total_minutes // self.minutes_per_hour) % self.hours_per_day
        new_minutes = total_minutes % self.minutes_per_hour
        self.current_time['hours'] = new_hours
        self.current_time['minutes'] = new_minutes
        print(f"Added {minutes} minutes. New time is {new_hours:02}:{new_minutes:02}")

    def display_time(self):
        hours = self.current_time['hours']
        minutes = self.current_time['minutes']
        print(f"Current time: {hours:02}:{minutes:02}")


class ConventionalClock(Clock):
```

```
    def __init__(self):
        super().__init__(24, 60)

    def describe_clock(self):
        print("This is a Conventional Clock with 24 hours per day and 60 minutes per hour.")


class AlteredClock(Clock):
    def __init__(self):
        super().__init__(12, 30)

    def describe_clock(self):
        print("This is an Altered Clock with 12 hours per day and 30 minutes per hour.")


print("--- Conventional Clock ---")
conventional_clock = ConventionalClock()
conventional_clock.set_time(10, 30)
conventional_clock.add_minutes(95)
conventional_clock.display_time()
conventional_clock.describe_clock()


print("\n--- Altered Clock ---")
```

```
altered_clock = AlteredClock()

altered_clock.set_time(5, 15)

altered_clock.add_minutes(50)

altered_clock.display_time()

altered_clock.describe_clock()
```